
การพัฒนาซอฟต์แวร์ และ แบบจำลองกระบวนการ

จัดทำโดย

1. อรปรียา อาสนทอง 6605001699
2. อภิญญา วงษ์ศรี 6605005344
3. กิมทีนา ลี 6705006069
4. สุทธิดา ทรงรัมย์ 6605005716
5. วิญญู สายแสง 6805007405

วงจรการพัฒนาซอฟต์แวร์

Software Development Life Cycle (SDLC)

SDLC คืออะไร?

กระบวนการทำงานมาตรฐานที่ช่วยควบคุมการออกแบบ สร้าง และติดตั้งระบบซอฟต์แวร์ ให้บรรลุคุณภาพสูงสุด ภายใต้กรอบเวลาและงบประมาณอย่างสมเหตุสมผล

- ✓ วิเคราะห์ปัญหาเป็นระบบ: เริ่มคิดอย่างรอบคอบรอบด้าน
- ✓ ลดความเสี่ยงโครงการ: มีขั้นตอนตรวจสอบชัดเจนก่อนลงมือโค้ดจริง
- ✓ คุณภาพตรงเป้าหมาย: ตรวจวัดและบำรุงรักษาง่ายในอนาคต



7 ขั้นตอนการทำงานที่เป็นระบบ (SDLC Process)

วงจรชีวิตการทำงานมีขั้นตอนลำดับเหตุผลที่เชื่อมประสานกันตั้งแต่ต้นจนจบ ดังนี้

01

Planning
(วางแผน)

02

Analysis
(วิเคราะห์)

03

Design
(ออกแบบ)

04

Develop
(พัฒนาโค้ด)

05

Testing
(ทดสอบ)

06

Deploy
(ติดตั้งจริง)

07

Maintain
(บำรุงรักษา)

ขั้นตอนการดำระบบ (ช่วงเริ่มต้น: 1 - 4)

01 Planning

วิเคราะห์งบประมาณ ระยะเวลา
โครงการ ความเป็นไปได้ และประเมิน
ความต้องการทั่วไปเพื่อเริ่มต้นโครงการ

- ประเมินงบประมาณ
- จัดสรรทีมพัฒนา
- **ผลลัพธ์:** Project Plan

02 Analysis

เก็บรวบรวม Requirement จากผู้ใช้
อย่างเจาะลึกเพื่อกำหนดฟังก์ชันสำคัญ
ของระบบ

- สัมภาษณ์ผู้ใช้จริง
- วิเคราะห์ข้อจำกัด
- **ผลลัพธ์:** เอกสาร SRS

03 Design

ออกแบบโครงสร้าง ซอฟต์แวร์
ฐานข้อมูล ตลอดจนจำลอง UI/UX ก่อน
การเริ่มเขียนโค้ด

- ER-Diagram & UML
- โครงสร้าง Architecture
- **ผลลัพธ์:** UI Prototype

04 Development

โปรแกรมเมอ์นำผลลัพธ์การออกแบบ
มาเริ่มพัฒนา เขียนโค้ดเชื่อมต่อระบบ

- และสร้างระบบฐานข้อมูล
- Frontend & Backend
 - เชื่อม API ระบบ
 - **ผลลัพธ์:** ซอฟต์แวร์พร้อมใช้

ขั้นตอนปลายน้ำ (5 - 7) และประโยชน์ของ SDLC

ช่วงท้ายของโครงการ (Steps 5-7)

- ✓ **05. Testing:** การทดสอบระบบหาข้อผิดพลาด (Bugs) โดยละเอียด เช่น UAT, Integration Testing
- ✓ **06. Deployment:** การติดตั้งขึ้น Server / Cloud การอบรม และถ่ายโอนข้อมูล
- ✓ **07. Maintenance:** การแก้ Bug, อัปเดตแพตช์ระบบความปลอดภัย และปรับปรุงฟังก์ชันรองรับตลาด

* ประโยชน์มหาศาลของการทำ SDLC

- ✓ เป็นขั้นตอนชัดเจน: ทุกทีมมีความเข้าใจขอบเขตงานตรงกัน
- ✓ ควบคุมควบคุมต้นทุนดี: ป้องกันไม่ให้งบบานปลายระหว่างพัฒนา
- ✓ ได้งานที่มีคุณภาพ: ตรวจพบความเสียหายก่อนปล่อยจริง
- ✓ ง่ายต่อการบำรุงรักษา: มีเอกสารคู่มือสนับสนุนระยะยาว

แบบจำลองกระบวนการพัฒนาระบบ

Software Process Models

Waterfall Model vs. Incremental Model

Waterfall Model (แบบจำลองน้ำตก)

ทำงานเป็นทอดๆ ตามลำดับเสร็จทีละขั้นแบบเรียงตัว ไม่ควรย้อนหลัง

- ✓ **ข้อดี:** บริหารง่าย มีโครงสร้างชัดเจน เอกสารแน่นหนา
- ✓ **ข้อเสีย:** เปลี่ยนไขข้อตกงยาก, ลูกค้าเห็นผลงานช้า
- ✓ **เหมาะสำหรับ:** งานราชการ, ระบบบัญชี, ระบบการเงิน

Incremental Model (แบบค่อยเพิ่มขึ้น)

แบ่งโปรเจกต์เป็นส่วนย่อย พัฒนาและทยอยปล่อยใช้งานจริงเป็นรอบๆ

- ✓ **ข้อดี:** ส่งงานขึ้นแรกเร็ว, ปรับตามความต้องการตลาดดี
- ✓ **ข้อเสีย:** ต้องวางสถาปัตยกรรมโครงสร้างหลักให้แน่นแต่แรก
- ✓ **เหมาะสำหรับ:** E-commerce, เว็บไซต์, แอปพลิเคชันมือถือ

RAD Model vs. Prototyping Model

RAD Model (เน้นความเร็ว)

เน้นการสร้างและปรับปรุงอย่างด่วนด้วยความช่วยเหลือของเครื่องมือสร้างโค้ดอัตโนมัติ

- ✓ **ข้อดี:** ระบบเสร็จไว, ปรับแต่งยืดหยุ่นสูง
- ✓ **ข้อเสีย:** สิ้นเปลืองงบ, ต้องการผู้เชี่ยวชาญฝีมือสูง
- ✓ **เหมาะสำหรับ:** Web App, ระบบธุรกิจขนาดกลาง

Prototyping Model (เน้นแบบจำลอง)

สร้าง mock-up เพื่อทดสอบความต้องการจนผู้ใช้พอใจค่อยเริ่มทำระบบหลังบ้านจริง

- ✓ **ข้อดี:** ความต้องการชัดเจน, ลดปัญหาขัดแย้งกับผู้ใช้
- ✓ **ข้อเสีย:** การประเมินงานผิดพลาด, ปรับแก้หลายรอบ
- ✓ **เหมาะสำหรับ:** นวัตกรรมใหม่, โครงการที่มีความไม่แน่นอนสูง

Spiral, Concurrent & Unified Process Models

Spiral

แบบจำลองแบบก้นหอย

ทำงานซ้ำแบบวนรอบผสมผสาน โดยเน้นเรื่อง "การวิเคราะห์ความเสี่ยง (Risk Analysis)" ในระดับสูงในทุกรอบวงโคจร

เหมาะกับ: ระบบการบิน ทหาร และธนาคาร

Concurrent

การพัฒนาไปพร้อมกัน

แต่ละทีมวิเคราะห์ ออกแบบ เขียนโค้ดคู่ขนานกันได้ โดยมีสถานะงานควบคุมของตนเอง ช่วยลดระยะเวลาภาพรวมลงได้มาก

เหมาะกับ: Cloud System และ Enterprise ใหญ่

UP Model

กระบวนการยูนิฟายด์

เน้นการออกแบบเชิงวัตถุ (Object-Oriented) โดยอิงความต้องการหลักตาม Use Case ของโครงสร้าง และแบ่งเป็น 4 ระยะ

เหมาะกับ: ระบบขนาดใหญ่ที่ใช้ UML นำร่อง

Capability Maturity Model Integration

กรอบการประเมินมาตรฐานคุณภาพองค์กร (CMMI)

มาตรฐานวุฒิภาวะ 5 ระดับ (CMMI Maturity Levels)

- | | |
|----------------|--|
| Level 5 | Optimizing (ปรับปรุงต่อเนื่อง)
ปรับวิธีการทำงานให้มีเทคโนโลยีและการพัฒนานวัตกรรมต่อเนื่องโดยใช้สถิติตามจุดบกพร่องที่พบ |
| Level 4 | Quantitatively Managed (เชิงสถิติ)
ใช้ตัวเลขและสถิติ (Metrics) ในการควบคุมความเร็วและคุณภาพซอฟต์แวร์อย่างเป็นวิทยาศาสตร์ |
| Level 3 | Defined (มาตรฐานองค์กร)
ทุกโครงการใช้คู่มือแนวปฏิบัติเป็นมาตรฐานเดียวกัน ไม่ใช่ต่างโครงการต่างจัดการ |
| Level 2 | Managed (จัดการโปรเจกต์)
มีการควบคุมต้นทุน ความปลอดภัย และ Requirement ของผู้ใช้ในโครงการอย่างมีระบบย้อนรอยได้ |
| Level 1 | Initial (เริ่มต้น-ไร้ระบบ)
การทำงานพึ่งพาคนเก่งเฉพาะตัว (Individual hero) ไม่มีมาตรฐานรองรับล่วงหน้า |

การประยุกต์ใช้ CMMI ในยุคปัจจุบัน

ข้อดี และ ข้อเสีย

ข้อดี: ตรวจสอบหาความผิดพลาดล่วงหน้าได้ดี เสริมสร้างเครดิตความน่าเชื่อถือสำหรับการรับงานประมูลระดับรัฐหรือสากล

ข้อเสีย: กลายเป็นอุปสรรคทางเอกสาร (Bureaucracy) เสียเวลาทำรายงานมากกว่าเขียนโปรแกรมจริง ไม่เหมาะกับทีมขนาดเล็กหรือ Startup

การพัฒนาแบบ Hybrid ในยุคใหม่

- ✓ **Agile-CMMI Hybrid:** ทำงานฉับไวด้วย Scrum ในแต่ละวัน แต่ควบคุมผลงานภาพรวมด้วยระบบตรวจสอบแบบ CMMI
- ✓ **Automated Metrics:** ใช้ตัวช่วยอย่าง Jira, Git, SonarQube เพื่อเก็บคะแนนสถิติอัตโนมัติ โดยไม่ต้องรบกวนเวลาเขียนโค้ด
- ✓ **CMMI V2.0:** มุ่งเน้นไปที่ผลสัมฤทธิ์ทางธุรกิจจริง (Performance) มากกว่าพิธีรีตอง

มีคำถามเพิ่มเติมหรือไม่?

ขอบคุณสำหรับการรับฟังและการมีส่วนร่วม

แหล่งข้อมูลอ้างอิง:

Foxbith Blog • Eastgate Software • GeeksforGeeks Software
Engineering Guide

Image Sources



<https://static.vecteezy.com/system/resources/thumbnails/046/700/325/small/abstract-technology-blue-circular-system-cyber-futuristic-on-white-design-modern-creative-background-vector.jpg>

Source: www.vecteezy.com



<https://www.kore1.com/wp-content/uploads/2026/04/salesforce-developer-interview-questions-featured-kore1-768x320.webp>

Source: www.kore1.com
